AI-Driven Retail Data Analytics Platform for Anomaly Detection and Predictive Insights

Abstract

In the modern retail environment, large volumes of transactional data are generated on a daily basis. Traditional data analytics approaches require manual inspection and predefined rules, making them inefficient in identifying unexpected patterns and future trends. This project presents an AI-Driven Retail Data Analytics Platform that automates data ingestion, cleaning, anomaly detection, predictive analytics, and insight generation. Using machine learning techniques such as KNN Imputation, Isolation Forest, and Long Short-Term Memory (LSTM) networks, the system detects anomalous transactions, forecasts future sales, and converts analytical results into human-readable insights. The platform integrates these capabilities into an interactive dashboard, enabling data-driven decision-making with minimal human intervention.

---

1. Introduction

Retail organizations rely heavily on data analytics to understand customer behavior, monitor sales performance, and detect irregular activities. However, conventional analytics tools primarily focus on visualization and require analysts to manually interpret results. With the increasing complexity and volume of data, there is a need for intelligent systems that can automatically analyze data, identify anomalies, predict future trends, and explain results in a comprehensible manner.

This project aims to bridge this gap by developing an AI-driven analytics platform that performs end-to-end data processing and analysis. The system not only processes raw retail data but also applies machine learning algorithms to extract meaningful insights and present them through an interactive dashboard.

---

2. Problem Definition

Traditional retail data analysis systems suffer from the following limitations: - Manual data cleaning and preprocessing - Lack of automated anomaly detection - Limited predictive capabilities - Dependence on human interpretation of charts and metrics

The problem addressed in this project is the development of an automated analytics system that can: - Handle raw, messy retail data - Detect unusual transactions automatically - Predict future sales trends - Generate human-readable insights without manual analysis

---

3. Objectives

The main objectives of this project are: - To design a robust data ingestion pipeline for retail datasets - To automate data cleaning using machine learning techniques - To detect anomalies in retail transactions - To perform time-series forecasting of sales using deep learning - To generate natural language insights from analytical results - To present results through an interactive and user-friendly dashboard

---

4. Dataset Description

The dataset used in this project is a synthetic retail store sales dataset designed to simulate real-world retail transactions. It contains 12,575 records with intentional inconsistencies such as missing values and irregular patterns.

Key Attributes:

- Transaction ID
- Customer ID
- Product Category
- Item Name
- Price Per Unit
- Quantity
- Total Spent
- Discount Applied
- Payment Method
- Location
- Transaction Date

Synthetic data was chosen to avoid privacy concerns while preserving realistic retail characteristics.

---

5. System Architecture

The system architecture consists of the following layers:

1. **Data Ingestion Layer**: Loads raw CSV data, validates schema, parses date fields, and stores unmodified data in a SQLite database.
2. **Data Cleaning Layer**: Applies machine learning-based preprocessing techniques to handle missing values and detect anomalies.
3. **Analytics and Prediction Layer**: Aggregates data, performs trend analysis, and predicts future sales using an LSTM model.
4. **Insight Generation Layer**: Converts analytical and predictive outputs into human-readable insights using rule-based natural language generation.
5. **Presentation Layer**: Displays metrics, charts, predictions, and insights through an interactive Streamlit dashboard.

---

6. Methodology

6.1 Data Ingestion

Raw retail data is ingested from a CSV file. Schema validation ensures the presence of required columns. Transaction dates are parsed, and the raw data is stored in a SQLite database to preserve data integrity and reproducibility.

6.2 Automated Data Cleaning

- **Missing Value Handling**: KNN Imputation is used to estimate missing numerical values based on nearest neighbors.
- **Anomaly Detection**: Isolation Forest is applied to detect unusual transactions. Anomalies are flagged rather than removed, as they may represent important business events.

6.3 Analytics and Prediction

Daily sales are aggregated from transaction-level data. An LSTM neural network is trained on historical sales data to predict future sales trends. Model performance is evaluated using Root Mean Squared Error (RMSE).

6.4 Insight Generation

Rule-based logic is used to generate natural language insights based on anomaly counts, sales trends, and prediction accuracy. These insights provide clear explanations for non-technical users.

---

7. Technologies Used

- **Programming Language**: Python
- **Data Processing**: Pandas, NumPy
- **Machine Learning**: Scikit-learn
- **Deep Learning**: TensorFlow (LSTM)
- **Database**: SQLite
- **Visualization & UI**: Streamlit, Plotly

---

8. Results and Analysis

- A total of 12,575 records were processed successfully.
- KNN Imputation handled missing values without discarding data.
- Isolation Forest detected 243 anomalous transactions.
- The LSTM model achieved a reasonable RMSE, indicating effective trend prediction.
- AI-generated insights summarized anomalies and future sales trends in plain English.

The dashboard provided a unified view of analytics, predictions, and insights, enabling effective decision support.

9. Limitations

- The dataset is synthetic and may not capture all real-world complexities.
- The prediction model uses a single variable (total sales) for forecasting.
- Rule-based insight generation may not capture highly complex relationships.

---

10. Future Scope

Future enhancements may include: - Integration of real-time data through APIs - Use of advanced forecasting models and ensemble techniques - Deployment on cloud platforms - Integration of large language models for more advanced insight generation

---

11. Conclusion

This project successfully demonstrates the application of artificial intelligence in retail data analytics. By automating data cleaning, anomaly detection, prediction, and insight generation, the system reduces reliance on manual analysis and enhances decision-making capabilities. The modular and scalable design ensures that the platform can be extended to real-world retail environments. Overall, the project highlights the effectiveness of AI-driven analytics in transforming raw data into actionable intelligence.

---

12. Appendix: Source Code

This section lists all the source code files used in the project along with their purposes. Each script is modular and performs a specific function within the system pipeline.

12.1 `ingest_data.py` – Data Ingestion Pipeline

Purpose: Loads raw retail data from CSV, validates schema, parses dates, and stores unmodified data in the SQLite database.

```
import pandas as pd
import sqlite3
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_PATH = BASE_DIR / "data" / "raw" / "retail_store_sales.csv"
DB_PATH = BASE_DIR / "database" / "retail_analytics.db"

print("Loading raw dataset...")
```

```
if not DATA_PATH.exists():
    raise FileNotFoundError(f"Dataset not found at {DATA_PATH}")

df = pd.read_csv(DATA_PATH)
print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}")

required_columns = [
    "Transaction ID",
    "Category",
    "Item",
    "Price Per Unit",
    "Quantity",
    "Total Spent",
    "Transaction Date"
]

missing_cols = [col for col in required_columns if col not in df.columns]
if missing_cols:
    raise ValueError(f"Missing required columns: {missing_cols}")

print("Schema validation passed.")

df["Transaction Date"] = pd.to_datetime(df["Transaction Date"], errors="coerce")
print("Date parsing complete.")

DB_PATH.parent.mkdir(parents=True, exist_ok=True)
conn = sqlite3.connect(DB_PATH)

df.to_sql(
    name="raw_sales_data",
    con=conn,
    if_exists="replace",
    index=False
)

conn.close()
print("Raw data successfully stored in database.")
```

12.2 `clean_data.py` – Automated Data Cleaning

Purpose: Performs machine-learning-based preprocessing using KNN Imputation and Isolation Forest for anomaly detection.

```
import pandas as pd
import sqlite3
from pathlib import Path
from sklearn.impute import KNNImputer
from sklearn.ensemble import IsolationForest

BASE_DIR = Path(__file__).resolve().parent.parent
DB_PATH = BASE_DIR / "database" / "retail_analytics.db"

conn = sqlite3.connect(DB_PATH)
df = pd.read_sql("SELECT * FROM raw_sales_data", conn)

numeric_cols = df.select_dtypes(include=["int64", "float64"]).columns.tolist()

imputer = KNNImputer(n_neighbors=5)
df[numeric_cols] = imputer.fit_transform(df[numeric_cols])

iso_forest = IsolationForest(n_estimators=100, contamination=0.02,
random_state=42)
df["anomaly_flag"] = iso_forest.fit_predict(df[numeric_cols])
df["anomaly_flag"] = df["anomaly_flag"].map({1: 0, -1: 1})

df.to_sql(
    name="cleaned_sales_data",
    con=conn,
    if_exists="replace",
    index=False
)

conn.close()
print("Cleaned data successfully stored in database.")
```

12.3 `analytics_predict.py` – Sales Analytics and Prediction

Purpose: Aggregates daily sales data and applies an LSTM model to predict future sales trends.

```
import pandas as pd
import sqlite3
import numpy as np
from pathlib import Path
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
```

```
BASE_DIR = Path(__file__).resolve().parent.parent
DB_PATH = BASE_DIR / "database" / "retail_analytics.db"

conn = sqlite3.connect(DB_PATH)
df = pd.read_sql("SELECT * FROM cleaned_sales_data", conn)
conn.close()

df["Transaction Date"] = pd.to_datetime(df["Transaction Date"])

daily_sales = (
    df.groupby("Transaction Date")["Total Spent"]
    .sum()
    .reset_index()
    .sort_values("Transaction Date")
)

sales_values = daily_sales["Total Spent"].values.reshape(-1, 1)
scaler = MinMaxScaler()
sales_scaled = scaler.fit_transform(sales_values)

def create_sequences(data, window=30):
    X, y = [], []
    for i in range(len(data) - window):
        X.append(data[i:i + window])
        y.append(data[i + window])
    return np.array(X), np.array(y)

X, y = create_sequences(sales_scaled)
split_index = int(len(X) * 0.8)

X_train, X_test = X[:split_index], X[split_index:]
y_train, y_test = y[:split_index], y[split_index:]

model = Sequential([
    LSTM(50, activation="relu", input_shape=(30, 1)),
    Dense(1)
])

model.compile(optimizer="adam", loss="mse")
model.fit(X_train, y_train, epochs=20, batch_size=32, verbose=1)

y_pred = model.predict(X_test)

y_test_inv = scaler.inverse_transform(y_test)
y_pred_inv = scaler.inverse_transform(y_pred)

rmse = np.sqrt(mean_squared_error(y_test_inv, y_pred_inv))
```

```
print(f"RMSE: {rmse}")

prediction_dates = daily_sales["Transaction Date"].iloc[-len(y_test_inv):]

pred_df = pd.DataFrame({
    "date": prediction_dates.values,
    "actual_sales": y_test_inv.flatten(),
    "predicted_sales": y_pred_inv.flatten()
})

conn = sqlite3.connect(DB_PATH)
pred_df.to_sql("sales_predictions", conn, if_exists="replace", index=False)
conn.close()
```

12.4 `generate_insights.py` – AI Insight Generator

Purpose: Generates human-readable insights from anomalies and prediction results using rule-based logic.

```
import pandas as pd
import sqlite3
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DB_PATH = BASE_DIR / "database" / "retail_analytics.db"

conn = sqlite3.connect(DB_PATH)
cleaned_df = pd.read_sql("SELECT * FROM cleaned_sales_data", conn)
pred_df = pd.read_sql("SELECT * FROM sales_predictions", conn)

insights = []

anomaly_count = cleaned_df["anomaly_flag"].sum()
total_records = len(cleaned_df)

if anomaly_count > 0:
    insights.append(
        f"{anomaly_count} unusual transactions were detected out of
{total_records} records."
    )

avg_actual = pred_df["actual_sales"].mean()
avg_predicted = pred_df["predicted_sales"].mean()

if avg_predicted > avg_actual:
    insights.append("Sales are predicted to increase in the near future.")
```

```
else:
    insights.append("Sales may decline or remain stable in the near future.")

error = abs(avg_predicted - avg_actual)
insights.append(f"Average prediction deviation is approximately {error:.2f}
units.")

insight_df = pd.DataFrame({
    "insight_id": range(1, len(insights) + 1),
    "insight_text": insights
})

insight_df.to_sql("generated_insights", conn, if_exists="replace", index=False)
conn.close()
```

12.5 `dashboard.py` – Interactive Dashboard

Purpose: Displays analytics, predictions, anomalies, and AI-generated insights using Streamlit.

```
import streamlit as st
import pandas as pd
import sqlite3
from pathlib import Path
import plotly.express as px

st.set_page_config(page_title="AI-Driven Retail Analytics", layout="wide")
st.title("AI-Driven Retail Data Analytics Platform")

BASE_DIR = Path(__file__).resolve().parent.parent
DB_PATH = BASE_DIR / "database" / "retail_analytics.db"

conn = sqlite3.connect(DB_PATH)
raw_df = pd.read_sql("SELECT * FROM raw_sales_data", conn)
cleaned_df = pd.read_sql("SELECT * FROM cleaned_sales_data", conn)
pred_df = pd.read_sql("SELECT * FROM sales_predictions", conn)
insights_df = pd.read_sql("SELECT * FROM generated_insights", conn)
conn.close()

col1, col2, col3 = st.columns(3)
col1.metric("Total Records", len(raw_df))
col2.metric("Cleaned Records", len(cleaned_df))
col3.metric("Detected Anomalies", int(cleaned_df["anomaly_flag"].sum()))

anomaly_counts = cleaned_df["anomaly_flag"].value_counts().rename(index={0:
"Normal", 1: "Anomaly"})
```

```
st.bar_chart(anomaly_counts)

fig = px.line(pred_df, x="date", y=["actual_sales", "predicted_sales"])
st.plotly_chart(fig, use_container_width=True)

st.subheader("AI-Generated Insights")
for _, row in insights_df.iterrows():
    st.markdown(f"- {row['insight_text']}")
```

References

1. Han, J., Kamber, M., & Pei, J. (2011). Data Mining: Concepts and Techniques.
2. Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn and TensorFlow.
3. Scikit-learn Documentation.
4. TensorFlow Documentation.